In [2]:
```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

In [4]:
```
max_features = 10000   # Vocabulary size
maxlen = 100           # Maximum length of each review
batch_size = 32
```

In [6]:
```
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_features)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
```

In [8]:
```
x_train = pad_sequences(x_train, maxlen=maxlen)
x_test = pad_sequences(x_test, maxlen=maxlen)
```

In [10]:
```
model = Sequential()

# Embedding layer (converts word indices to dense vectors)
model.add(Embedding(input_dim=max_features, output_dim=128, input_length=maxlen))

# LSTM layer
model.add(LSTM(64, dropout=0.2, recurrent_dropout=0.2))

# Output layer (binary classification)
model.add(Dense(1, activation='sigmoid'))

# Build model properly before summary
model.build(input_shape=(None, maxlen))

# Display model architecture
model.summary()
```

```
C:\Users\vaibhav\anaconda3\Lib\site-packages\keras\src\layers\core\embedding.py:90: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

**Model: "sequential"**

| Layer (type) | Output Shape | |
|---|---|---|
| embedding (Embedding) | (None, 100, 128) | |
| lstm (LSTM) | (None, 64) | |
| dense (Dense) | (None, 1) | |

**Total params:** 1,329,473 (5.07 MB)

**Trainable params:** 1,329,473 (5.07 MB)

**Non-trainable params:** 0 (0.00 B)

In [12]:
```
model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)
```

In [14]:
```
history = model.fit(
    x_train,
    y_train,
    batch_size=batch_size,
    epochs=5,
    validation_data=(x_test, y_test)
)
```

```
Epoch 1/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 40s 47ms/step - accuracy: 0.7254 - loss: 0.5285 - val_a
ccuracy: 0.8346 - val_loss: 0.3722
Epoch 2/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 37s 47ms/step - accuracy: 0.8673 - loss: 0.3222 - val_a
ccuracy: 0.8384 - val_loss: 0.3686
Epoch 3/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 37s 47ms/step - accuracy: 0.9017 - loss: 0.2433 - val_a
ccuracy: 0.8226 - val_loss: 0.4046
Epoch 4/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 37s 48ms/step - accuracy: 0.9288 - loss: 0.1854 - val_a
ccuracy: 0.8360 - val_loss: 0.4107
Epoch 5/5
782/782 ━━━━━━━━━━━━━━━━━━━━ 38s 48ms/step - accuracy: 0.9484 - loss: 0.1384 - val_a
ccuracy: 0.8400 - val_loss: 0.4782
```

In [16]:
```
score, acc = model.evaluate(x_test, y_test, batch_size=batch_size)

print("Test Loss:", score)
print("Test Accuracy:", acc)
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 6s 8ms/step - accuracy: 0.8394 - loss: 0.4881
Test Loss: 0.47819358110427856
Test Accuracy: 0.8400400280952454
```

In [18]:
```
# Predict sentiment of first test review
prediction = model.predict(x_test[0].reshape(1, -1))

print("Predicted Sentiment (0 = Negative, 1 = Positive):", prediction[0][0])
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 337ms/step
Predicted Sentiment (0 = Negative, 1 = Positive): 0.06586646
```

In [ ]: